

STA 5635 HW 13

Palmer Swanson

April 16, 2026

Part a

Normalized guidewire image



Figure 1: Normalized Image with Guidewire almost horizontal

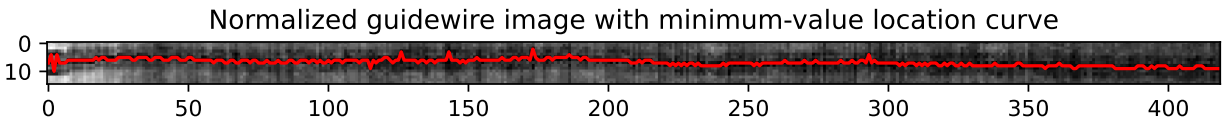


Figure 2: Normalized Image with Minimum Value locations in each column of the image overlaid

Part b

Based upon what we have learned in class, I started by going through what algorithms could NOT work here. Decision trees and random forests don't make sense; they are supervised learning. Logistic regression (classification) doesn't work either. Neural networks neither. We aren't clustering, so PCA isn't appropriate. Metric learning is close, but isn't exactly what I'm looking for.

There is a effectively a penalty term involved, but its not penalized loss functions as we learned them. Yesterday, Prof. Barbu talked about using the Viterbi algorithm, and I realized that it can be thought of as a Hidden Markov Model, estimated with the Viterbi algorithm. My reasoning for this is that we can view the rows of the iamge as states, the columns as observations, $I(c, i)$ as the emission cost, and the penalty term involving alpha acting as a transition cost.

Part c

α	Minimum $f(c)$
2	322.0000
15	1232.0000

Part d

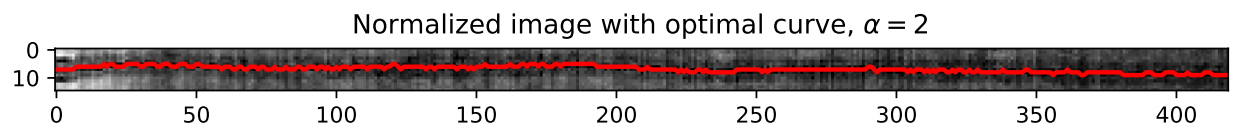


Figure 3: Normalized Image with Approximation of Guidewire c , $\alpha = 2$

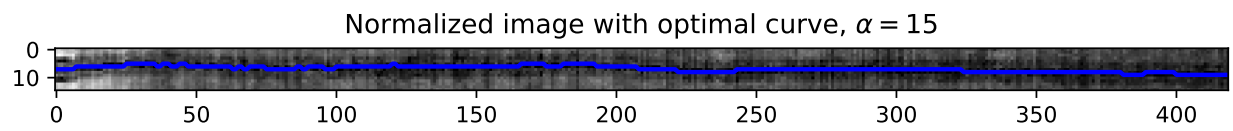


Figure 4: Normalized Image with Approximation of Guidewire c , $\alpha = 15$

Code

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 from PIL import Image
4 import pandas as pd
5
6 #need to load image and convert to array
7 image = Image.open("assignments/hw13/data/gw.bmp")
8 img_array = np.array(image)
9
10 #part a
11
12 #subtract the minimum of each column
13 column_minimums = np.min(img_array, axis=0)
14 image_norm = img_array - column_minimums
15
16 #display the normalized image
17 plt.figure(figsize=(8, 4))
18 plt.imshow(image_norm, cmap="gray")
19 plt.title("Normalized_guidewire_image")
20 plt.tight_layout()
21 plt.axis("off")
22 plt.savefig("assignments/hw13/figures/normalized_image.pdf", dpi=400,
23           bbox_inches="tight")
24 #plt.show()
25 plt.close()
26
27 #need a plot with a curve through the minima
28 #need to the get the locations of the minimum values from each column
29 min_locations = np.argmin(img_array, axis=0)
30 cols = np.arange(img_array.shape[1])
31 rows = min_locations
32
33 plt.figure(figsize=(8, 4))
34 plt.imshow(image_norm, cmap="gray")
35 plt.plot(cols, rows, color="red")
36 plt.title("Normalized_guidewire_image_with_minimum-value_location_curve")
37 plt.tight_layout()
38 plt.savefig("assignments/hw13/figures/gw_with_curve.pdf", dpi=400,
39           bbox_inches="tight")
40 #plt.show()
41 plt.close()
42
43 #parts
44 results = {}
```

```

44 alphas = [2, 15]
45
46 #get dimensions of normalized image
47 rows, columns = image_norm.shape
48
49 #rows are states here
50 states = np.arange(rows)
51
52 #I(i,j) is the pixel intensity at row i and column j of the image
53 #Viterbi algorithm, similar to what we did in the HMM homework
54 #difference is we are trying to minimize, whereas Viterbi is a maximization
   problem
55 #https://en.wikipedia.org/wiki/Viterbi_algorithm#Pseudocode
56 for alpha in alphas:
57
58     #minimum cost of path ending in state i column j
59     prob = np.full((columns, rows), np.inf)
60
61     #previous state in the optimal path to state i column j
62     prev = np.zeros((columns, rows), dtype=int)
63
64     #initialize first column
65     for s in states:
66         prob[0, s] = image_norm[s, 0]
67
68     #now need the recursion step
69     for column in range(1, columns):
70         for s in states:
71             for r in states:
72                 new_cost = prob[column - 1, r] + alpha * (s - r)**2 +
image_norm[s, column]
73
74                 if new_cost < prob[column, s]:
75                     prob[column, s] = new_cost
76                     prev[column, s] = r
77
78     path = np.zeros(columns, dtype=int)
79     path[columns - 1] = np.argmin(prob[columns - 1, :])
80
81     #go backwards now
82     for t in range(columns - 2, -1, -1):
83         path[t] = prev[t + 1, path[t + 1]]
84
85     #minimum value
86     f_min = prob[columns - 1, path[columns - 1]]
87
88     #store results

```

```

89     results[alpha] = {
90         "f_min": f_min,
91         "curve": path
92     }
93
94 #create dataframe to save reporting to
95 df = pd.DataFrame({
96     "alpha": [2, 15],
97     "f_min": [results[2]["f_min"], results[15]["f_min"]]
98 })
99 df.columns = [r"$\alpha$", r"Minimum  $f(c)$ "]
100
101 #save table to latex
102 df.to_latex(
103     "assignments/hw13/output/results_table.tex",
104     index=False,
105     float_format="%.4f",
106     escape=False
107 )
108
109 #plotting is pretty easy now
110 colors = {2: "red", 15: "blue"}
111
112 for alpha in [2, 15]:
113     plt.figure(figsize=(8, 4))
114     plt.imshow(image_norm, cmap="gray")
115     plt.plot(np.arange(image_norm.shape[1]), results[alpha]["curve"],
116             color=colors[alpha], linewidth=2)
117     plt.title(fr"Normalized  $\text{image}$  with optimal  $\text{curve}$ ,  $\alpha = \{alpha\}$ ")
118     plt.tight_layout()
119     plt.savefig(f"assignments/hw13/figures/gw_curve_alpha_{alpha}.pdf",
120               dpi=400, bbox_inches="tight")
121     #plt.show()
122     plt.close()
123
124 #images are really wide and long. Gotta be a way to change this shape

```